Universidad Tecnológica de Bolívar

Ingeniería de Sistemas y Computación

Diseño, implementación y pruebas de un agente inteligente

ReviewPulse-Agent — Agente de análisis de sentimientos en reseñas de hotel

Presentado por:

Ellen Nicole Ordoñez Chavez — T00079685

Sara Bastidas Triana — T00081674

Curso: Inteligencia Artificial

Docente: Edwin Alexander Puertas del Castillo

Cartagena, octubre de 2026

Tabla de cambios frente a la entrega anterior

La tabla resume qué cambió en cada sección del documento de la Tarea 2 y por qué. El texto original se conserva salvo donde se indica.

Sección	Qué decía	Qué dice ahora	Por qué cambió
Portada	Un autor; título «Diseño inicial»; 24 de agosto de 2026.	Pareja Ordoñez–Bastidas; título «Diseño, implementación y pruebas»; octubre de 2026.	La Tarea 3 se entrega en parejas y el documento ahora incluye la implementación.
0 (nueva)	—	Matriz de decisión ponderada, sensibilidad y justificación.	Requisito de la Fase 1.
1. Introducción	Errores de ortografía y puntuación («problematica», enumeración sin dos puntos).	Texto corregido; se aclara que el piloto se valida en un entorno simulado.	Redacción y alcance real de esta entrega.
2. Tipo de agente	Basado en modelos y orientado a objetivos, sin indicar cómo se realiza cada rasgo.	Mismo tipo. El estado interno es la memoria de trabajo y el repositorio SQLite; las metas se alcanzan con un planificador STRIPS con búsqueda en anchura.	Coherencia entre tipo de agente e implementación.
3. Tipos de ambiente	Clasificación en texto corrido.	Tabla resumen y, para cada propiedad, cómo la reproduce el simulador. El tiempo se discretiza en ciclos de 6 horas.	Trazabilidad con el simulador (Fase 2).
4. Entorno	Plataformas reales TripAdvisor y Google.	Siguen siendo las fuentes del diseño; en esta entrega se simulan con conectores que imitan su API, sus caídas y sus retrasos de publicación.	No hay acceso a las API dentro de sus términos de uso para un proyecto de curso.
5. Entradas	Cuatro entradas sin tipo de dato ni rango.	Tabla con tipo, rango y origen; se agregan el reloj del ciclo y la disponibilidad de cada plataforma.	La implementación necesita esa especificación.
6. Salidas	Cuatro salidas sin condición de disparo.	Tabla con actuador, condición y efecto; se agregan la marca de datos incompletos y la alerta tardía.	Coherencia con el código.
7. Medidas de desempeño	Metas sin fórmula; la detección decía «cualquier variación que supere el umbral», lo cual se cumple por definición.	Cada medida con fórmula, umbral y conjunto de prueba. M1 se mide por separado para polaridad y tema en un conjunto de control. M2 pasa a ser el porcentaje de incidentes reales alertados en 24 h (≥ 90 %).	Medibilidad: una medida que se cumple por definición no se puede probar.
8. Componentes IBM	«una simple regla de aplicada»; modelo tipo BERT.	Redacción corregida; el razonamiento es una prueba binomial con corrección de Bonferroni, aumento mínimo y mínimo de reseñas; el modelo es TF-IDF + léxico (BERT queda como reemplazo).	Coherencia con la implementación (ver 9.6).
9 (nueva)	—	Desarrollo del agente: arquitectura,	Requisito de la Fase 2.

Sección	Qué decía	Qué dice ahora	Por qué cambió
		tecnologías, entradas, salidas, diagramas de secuencia, diferencias, pruebas, limitaciones y enlaces.	
10. Conclusiones	Sección 9, sin resultados.	Renumerada y basada en los resultados de las pruebas.	Requisito de la Tarea 3.
Referencias	Tres referencias.	Se agregan scikit-learn, Streamlit y la referencia de prueba binomial.	Nuevas herramientas y métodos.

0. Selección justificada del agente

La pareja tenía asignados dos diseños: **ReviewPulse-Agent** (análisis de sentimiento en reseñas de hotel, agente A) y **RetainIQ-Agent** (predicción de rotación de empleados, agente B). Los pesos se fijaron antes de puntuar, con los valores sugeridos por la guía, porque reflejan bien la restricción principal del equipo: construir y probar el agente en el tiempo del curso. El cálculo completo está en el repositorio (docs/matriz_decision.py), de modo que cualquier persona puede reproducirlo.

Criterio	Peso (w)	A (1–5)	A: evidencia	B (1–5)	B: evidencia
Viabilidad técnica	25 %	4	4: sabemos implementar 6 de los 7 componentes IBM (todos salvo aprendizaje en línea); clasificación de texto con scikit-learn ya vista en clase.	3	3: sabemos implementar 5 de 7; la función de utilidad y la recalibración supervisada con revisión de equidad son nuevas para el equipo.
Disponibilidad de datos o entorno	20 %	4	4: existe un corpus de reseñas de TripAdvisor en español (IberLEF 2021) y el flujo semanal se puede simular con incidentes conocidos.	3	3: el dataset IBM HR de Kaggle (1.470 empleados) es de un solo corte; no trae trayectoria mensual ni fecha de renuncia, que habría que inventar.
Medibilidad del desempeño	20 %	5	5: 4 de 4 medidas se pueden escribir con fórmula y umbral, y el simulador conoce la verdad (incidentes y etiquetas).	3	3: 1 medida (precisión ≥ 75 %); requiere saber quién renunció en los 3 meses siguientes, dato que el dataset no tiene.
Complejidad de IA	15 %	3	3: basado en modelos y orientado a metas (estado interno + planificación).	5	5: basado en utilidad con aprendizaje supervisado periódico.
Costo y recursos	10 %	5	5: 0 COP; Python y bibliotecas libres, sin API pagas ni GPU.	5	5: 0 COP; mismas condiciones.
Valor del problema	10 %	3	3: gerente de un hotel piloto; detecta quejas en 24 h en lugar de leerlas a mano cada semana.	4	4: área de RR. HH.; anticipar renuncias evita costos de reemplazo, que suelen ser altos.
Puntaje ponderado S	100 %	4,05	$S = \Sigma\, w_i \times p_i$	3,60	$S = \Sigma\, w_i \times p_i$

Cálculo para A: 0,25×4 + 0,20×4 + 0,20×5 + 0,15×3 + 0,10×5 + 0,10×3 = **4,05**. Para B: 0,25×3 + 0,20×3 + 0,20×3 + 0,15×5 + 0,10×5 + 0,10×4 = **3,60**.

Análisis de sensibilidad

El criterio más influyente es el que más pesa en la diferencia entre los dos agentes, $|w_i \times (p_i^A - p_i^B)|$: **medibilidad del desempeño** (0,20 × 2 = 0,40), seguido de complejidad de IA (0,30, a favor de B). Al subir o bajar su peso 10 puntos y repartir la diferencia entre los demás criterios en proporción a su peso, el resultado es:

Cambio en medibilidad	Peso nuevo	S (A) ReviewPulse	S (B) RetainIQ	Elegido
+10 puntos	30 %	4,169	3,525	ReviewPulse
-10 puntos	10 %	3,931	3,675	ReviewPulse

Se repitió la prueba con los seis criterios (±10 puntos cada uno, 12 combinaciones): ReviewPulse gana en todas. El caso más estrecho es subir complejidad de IA a 25 %, donde la ventaja baja a 0,16 puntos. La decisión no depende de un peso en particular.

Justificación

Se elige **ReviewPulse-Agent** con 4,05 frente a 3,60. Gana porque sus cuatro medidas de desempeño se pueden probar de forma objetiva contra una verdad conocida y porque su entorno se puede simular respetando cada propiedad declarada en la sección 3. RetainIQ depende de datos laborales mensuales y de renuncias futuras que no tenemos, y su única medida no se podría verificar sin inventar esos datos.

Qué se pierde al descartar RetainIQ-Agent: la parte de utilidad y de aprendizaje supervisado, que es la de mayor complejidad de IA, y un problema con impacto económico directo para Recursos Humanos. Para compensar en parte, ReviewPulse incorpora planificación por metas con búsqueda y una regla de decisión estadística, no solo condicionales.

1. Introducción y planteamiento del problema

Antes de reservar una habitación, gran parte de los huéspedes revisa lo que otras personas han escrito en plataformas como TripAdvisor o Google. Ese hábito deja un volumen de comentarios que crece de forma constante y que rara vez tiene una estructura ordenada: son textos libres, escritos con distinto nivel de detalle, que mezclan opiniones sobre la limpieza, el servicio, la ubicación o el precio sin seguir ningún criterio fijo. Revisar ese material a mano, reseña por reseña, se vuelve insostenible a medida que el hotel gana visibilidad, y además llega tarde: para cuando alguien del equipo nota un patrón de quejas, es posible que varias semanas de huéspedes ya hayan tenido una experiencia deficiente sin que nadie interviniera.

De esa problemática surge ReviewPulse-Agent, un agente inteligente que clasifica automáticamente cada reseña de un hotel piloto según su polaridad (positiva, negativa o neutra) y su tema principal, y que detecta cuándo el tono de esas reseñas se aparta de lo habitual. No es un problema que se resuelva con una búsqueda de palabras clave. Exige tres cosas: interpretar lenguaje natural con sus ambigüedades, mantener memoria de las semanas anteriores para poder comparar y decidir a tiempo cuándo una variación merece la atención del gerente. Interpretación, memoria y oportunidad son las que justifican un agente inteligente en lugar de un mecanismo de reglas fijas.

En esta entrega el agente se implementa y se prueba sobre un entorno simulado del hotel piloto (sección 9), porque no hay acceso a las API de las plataformas dentro de sus términos de uso para un proyecto de curso.

2. Tipo de agente inteligente

Siguiendo la clasificación de Russell y Norvig en *Artificial Intelligence: A Modern Approach*, ReviewPulse-Agent es un **agente basado en modelos con orientación a objetivos**. Es basado en modelos porque una sola reseña no dice casi nada sobre si algo está cambiando: para saber si un aumento de comentarios negativos es una anomalía, el agente compara lo que ve ahora contra lo que vio en semanas pasadas, y esa comparación solo es posible si conserva y actualiza una representación interna del comportamiento habitual del hotel. En la implementación, ese estado interno es la memoria de trabajo (conteos de la semana abierta) y el repositorio histórico (agregados semanales, alertas y reportes).

Es orientado a objetivos porque persigue metas explícitas: mantener los conteos al día, entregar el resumen semanal dentro de 24 horas y revisar semanas cerradas cuando llegan reseñas tardías. En cada ciclo el agente describe su situación con hechos (hay lote nuevo, la semana cerró, la plataforma responde) y un planificador busca la secuencia de acciones que alcanza la meta. Si la meta no es alcanzable todavía, por ejemplo porque una plataforma está caída y aún hay plazo, el plan cambia a esperar. Eso descarta tanto al agente reflejo simple como a uno basado en modelos sin metas.

No es un agente basado en utilidad, porque no pondera de forma continua distintos estados del entorno con una función de utilidad, ni un agente de aprendizaje en sentido estricto, porque el modelo de sentimiento se entrena antes del despliegue y el agente no ajusta sus parámetros mientras opera.

3. Tipos de ambiente

Dimensión	Clasificación	Justificación	Cómo lo reproduce el simulador
Observabilidad	Parcialmente observable	El agente solo ve el texto, la fecha, la plataforma y la calificación; no ve qué pasó en la estadía ni qué opinan quienes no escriben.	El estado operativo del hotel (incidentes por tema) existe en el simulador, pero el agente nunca lo lee.
Determinismo	Estocástico	Reseñas parecidas pueden clasificarse distinto por ironía o expresiones regionales; el volumen semanal es incierto.	Volumen Poisson, polaridad y tema aleatorios, ironía, errores de tipeo y caídas de plataforma.
Episodicidad	Secuencial	Notar un cambio solo tiene sentido frente a las semanas anteriores.	14 semanas encadenadas; cada semana se compara con las 6 anteriores.
Dinamismo	Dinámico	Las reseñas siguen publicándose mientras el agente procesa.	Cada reseña se vuelve visible con un retraso aleatorio de hasta 30 h; una reseña del domingo puede llegar el lunes, después del cierre.
Discretización	Discreto	Reseñas contables, categorías cerradas y cortes semanales.	El agente actúa en ciclos fijos de 6 horas.
Número de	Agente único	Las plataformas y el canal de notificación	Un solo agente; las plataformas son

Dimensión	Clasificación	Justificación	Cómo lo reproduce el simulador
agentes		son herramientas, no agentes con objetivos propios.	conectores.

4. Descripción del entorno

ReviewPulse-Agent opera en un espacio deliberadamente acotado: un único hotel piloto y las reseñas públicas que acumula en TripAdvisor y Google. Conviven tres actores: los huéspedes, que generan la materia prima del sistema cada vez que escriben un comentario; el gerente del hotel, que recibe los resúmenes semanales y las alertas y decide qué hacer con esa información; y las plataformas de reseñas, que son una fuente externa de datos que el agente no controla.

El entorno tiene límites claros. El agente solo puede recolectar información respetando los términos de uso de cada plataforma, depende de un modelo de sentimiento para el español de reseñas de hospedaje y su alcance no va más allá del hotel piloto. En un día cualquiera llega un flujo moderado de reseñas (unas 36 por semana en el simulador) con una distribución de opiniones estable. Pero también puede pasar que un problema de servicio dispare los comentarios negativos, que una plataforma quede inaccesible, que aparezcan reseñas en otro idioma o cargadas de ironía, o que la temporada alta traiga más huéspedes y más información para procesar. Los doce escenarios de prueba de la sección 9.7 cubren cada uno de estos casos.

5. Entradas y sensores

Todas las entradas se capturan por mecanismos digitales; no hay sensores físicos. La sección 9.3 detalla tipo, rango y un ejemplo real de cada una.

Entrada	Sensor (mecanismo)	Uso en la decisión
Texto de la reseña	Conector de cada plataforma: consulta de reseñas visibles desde el último cursor	Insumo del clasificador de polaridad y tema
Metadatos: identificador, fecha, plataforma, calificación de 1 a 5	Mismo conector, en la misma consulta	La fecha asigna la semana; la calificación se guarda en el reporte
Disponibilidad de cada plataforma	Resultado de la consulta (éxito o PlataformaNoDisponible)	Decide si esperar reintento o cerrar con datos incompletos
Historial del hotel	Consulta al repositorio SQLite	Línea base de comparación de las últimas 6 semanas
Parámetros de configuración	config.json o ajustes de las alertas de la interfaz	Umbrales de la regla de alerta
Reloj del ciclo	Planificador de ejecución cada 6 horas	Determina qué semanas cerraron y cuántas horas han pasado

6. Salidas y actuadores

Acción	Actuador	Condición que la dispara	Efecto en el entorno
Etiqueta de clasificación	Escritura en la tabla resenas (SQLite)	Cada reseña válida del lote	La reseña queda con polaridad, tema, confianza y marca de revisión manual si la confianza es menor de 0,55
Alerta al gerente	Notificador: correo SMTP o bandeja de salida JSONL	Al cierre semanal, si un tema supera la regla de alerta	El gerente recibe un mensaje en lenguaje natural con la proporción, el promedio y dos ejemplos
Alerta tardía	Notificador	Llegan reseñas atrasadas de una semana ya cerrada y la reevaluación supera la regla	Aviso marcado como tardío
Resumen semanal por tema	Generador de reportes (Markdown y JSON)	Semana cerrada, con margen de 12 h o en el último ciclo antes de 24 h	Reporte con conteos por tema y marca de datos incompletos si alguna plataforma falló
Registro histórico	Escritura en tablas semanas, alertas y reportes	Después de cada reporte	Línea base para las semanas siguientes y trazabilidad
Descarte de entrada	Escritura en la tabla descartadas	Texto vacío, otro idioma, HTML sin contenido o sin fecha	La reseña no entra a los conteos

7. Medidas de desempeño

Las medidas se reescribieron con fórmula, umbral y conjunto de prueba para que se puedan verificar de forma objetiva. Los umbrales se fijaron antes de ejecutar las pruebas (pruebas/umbrales.json) y no se modificaron después.

Medida	Fórmula	Umbral	Conjunto y plazo
M1 Exactitud de clasificación	Reseñas con polaridad correcta / reseñas evaluadas; igual para el tema	≥ 85 % en polaridad y ≥ 85 % en tema	Conjunto de control de 40 reseñas redactadas aparte del corpus de entrenamiento, con asistencia de IA (ver bitácora), nunca usado para ajustar el modelo; 5 reentrenamientos
M2 Detección de cambios en el sentimiento	Incidentes relevantes alertados ≤ 24 h tras el cierre / incidentes relevantes	≥ 90 %	12 escenarios × 5 semillas; el simulador sabe qué semanas tuvieron un incidente real
M3 Generación oportuna de reportes	Reportes entregados ≤ 24 h tras el cierre / semanas cerradas	100 %	Las mismas 60 corridas
M4 Errores de notificación	Alertas sin incidente relevante / alertas enviadas	< 10 %	Las mismas 60 corridas

M2 se reformuló porque la versión anterior («reconocer cualquier variación que supere el umbral») se cumple por definición: el agente siempre reconoce lo que su propio umbral marca. La versión nueva

compara las alertas contra incidentes reales conocidos por el simulador, que es lo que le importa al gerente.

8. Componentes de la arquitectura de IA agéntica (marco IBM)

De acuerdo con la arquitectura de IA agéntica descrita por IBM, los componentes de un agente se agrupan en siete áreas funcionales. Así las implementa ReviewPulse-Agent; la sección 9.1 indica el módulo de código de cada una.

Percepción y manejo de entradas

El método percibir() consulta cada seis horas las plataformas del hotel en busca de reseñas nuevas y recoge, junto con el texto, sus metadatos. Antes de que el contenido llegue al clasificador pasa por una limpieza: se quitan etiquetas HTML y entidades, se verifica el idioma, se descartan textos vacíos o sin fecha y se normaliza el formato.

Planificación y descomposición de tareas

La meta de fondo (tener al gerente informado sobre cómo perciben los huéspedes al hotel) se divide en metas concretas y en acciones con precondiciones y efectos: limpiar el lote, clasificarlo, actualizar los conteos, comparar con el historial, decidir la alerta, generar el reporte y registrar el historial. Un planificador tipo STRIPS con búsqueda en anchura encuentra en cada ciclo la secuencia que lleva a la meta activa.

Memoria

En el corto plazo, mientras la semana sigue abierta, el agente mantiene en memoria de trabajo el conteo de reseñas por tema y polaridad. En el largo plazo guarda en un repositorio SQLite las reseñas etiquetadas, los agregados de semanas anteriores, los resúmenes entregados y las alertas enviadas. Esa memoria le da contra qué comparar y deja trazabilidad de lo que hizo y por qué.

Razonamiento y toma de decisiones

Decidir si algo amerita una alerta no es una regla aplicada a ciegas. Para cada tema, el agente compara la proporción de reseñas negativas de la semana con la de las seis semanas anteriores mediante una prueba binomial con corrección de Bonferroni (porque se evalúan cuatro temas a la vez), y exige además un aumento mínimo de la proporción y un mínimo de reseñas negativas. La confianza de cada clasificación también entra al razonamiento: las reseñas con confianza menor de 0,55 cuentan la mitad y se marcan para revisión manual.

Llamada de herramientas y acción (tool calling)

El agente se apoya en herramientas externas para cada parte del proceso: el modelo de sentimiento para clasificar polaridad y tema, los conectores hacia las plataformas, la base de datos SQLite para leer y escribir el historial, el generador de reportes y el notificador que envía la alerta al gerente.

Comunicación

Los resultados se presentan para un usuario no técnico: el resumen semanal por tema, como reporte y como panel en la interfaz, y alertas en lenguaje natural que dicen qué se detectó, cuánto subió frente al promedio y dan dos ejemplos de reseñas.

Aprendizaje y adaptación

En esta versión el modelo de sentimiento se entrena antes de ponerse en marcha y el agente no lo ajusta mientras opera. Queda como paso a futuro usar las reseñas que el gerente revise manualmente para reentrenar el modelo cada cierto tiempo.

9. Desarrollo del agente

9.1 Arquitectura

El agente es un paquete de Python (reviewpulse) con un núcleo, ReviewPulseAgent, que ejecuta cada seis horas de tiempo simulado el ciclo percepción → decisión → acción. El entorno, que en producción serían las API de las plataformas, es un simulador con la misma interfaz que tendría un conector real. La figura 1 muestra los componentes y sus conexiones.

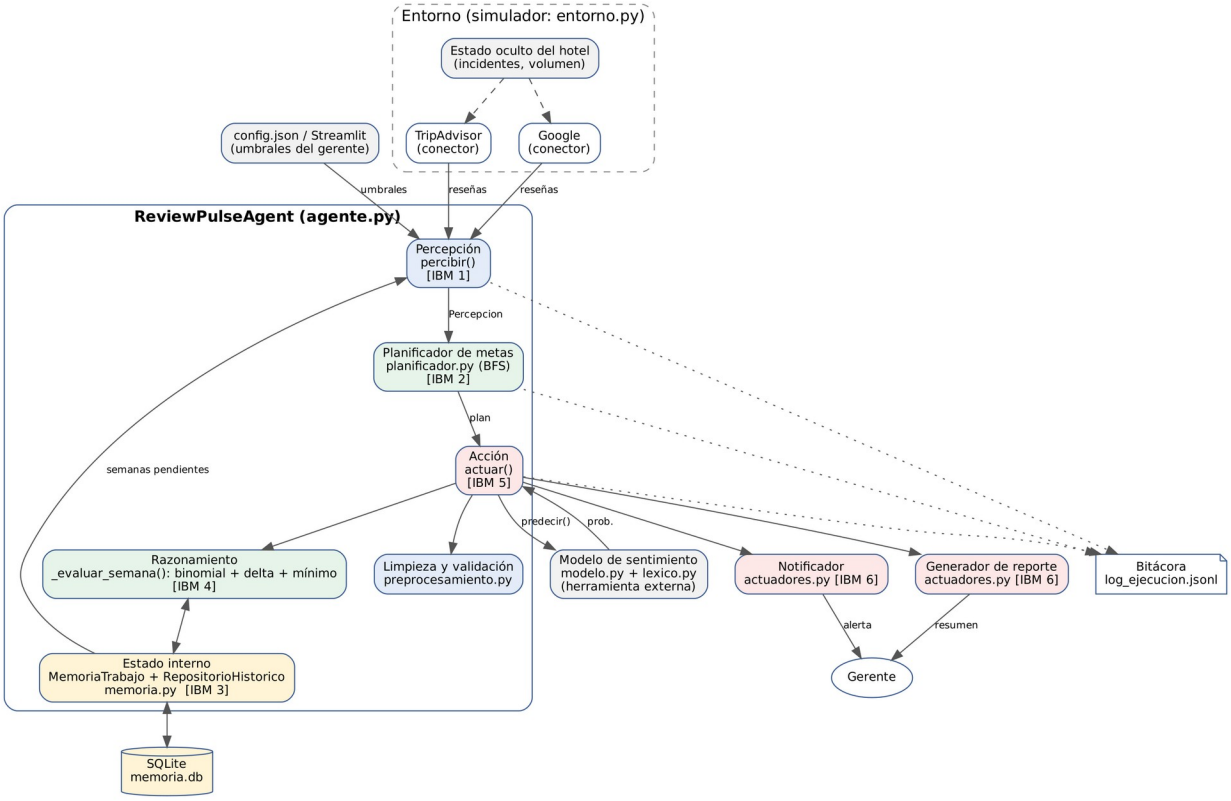


Figura 1. Diagrama de componentes de ReviewPulse-Agent. Entre corchetes, el componente IBM que implementa.

El ciclo está en el método paso() de reviewpulse/agente.py:

```
def paso(self, ahora):
    per  = self.percibir(ahora)   # PERCEPCIÓN: conectores, historial, config
    plan = self.decidir(per)      # DECISIÓN: hechos -> meta -> plan (BFS)
    self.actuar(plan, per)        # ACCIÓN: etiquetas, alerta, reporte, registro
```

Componente IBM	Módulo y función	Estado
1. Percepción y manejo de entradas	agente.percibir(); preprocesamiento.validar(), limpiar(), detectar_idioma(); entorno.consultar()	Implementado
2. Planificación y descomposición	planificador.planificar() (STRIPS + BFS); agente.decidir() y agente._hechos()	Implementado
3. Memoria	memoria.MemoriaTrabajo (corto plazo); memoria.RepositorioHistorico (SQLite, largo plazo)	Implementado
4. Razonamiento y toma de	agente._evaluar_semana(): prueba binomial con Bonferroni,	Implementado

Componente IBM	Módulo y función	Estado
decisiones	aumento mínimo, mínimo de negativas; umbral de confianza	
5. Llamada de herramientas y acción	agente.actuar() y métodos _a_*; modelo.ClasificadorSentimiento.predecir(); actuadores.Notificador, GeneradorReporte	Implementado
6. Comunicación	actuadores.Notificador.redactar() (alerta en lenguaje natural); GeneradorReporte.generar(); app/streamlit_app.py	Implementado
7. Aprendizaje y adaptación	modelo.entrenar_modelo_base() se ejecuta antes del despliegue	No implementado en línea: el diseño lo deja como trabajo futuro (sección 8) y reentrenar sin etiquetas del gerente propagaría los errores del propio modelo

9.2 Tecnologías

Tecnología	Versión	Para qué se usó	Por qué frente a una alternativa
Python	3.13	Lenguaje de todo el agente, el simulador y las pruebas	Frente a Java: el equipo lo maneja y tiene el ecosistema de aprendizaje automático
scikit-learn	1.9.1	TF-IDF de palabras y caracteres, regresión logística multinomial	Frente a un BERT (transformers): corre sin GPU ni descarga de modelos, entrena en 4 s y da probabilidades calibradas
Léxico propio (lexico.py)	1.0	Rasgos de polaridad y tema con manejo de negación	Frente a un léxico genérico en inglés (VADER): está escrito para español de reseñas de hospedaje
NumPy	2.5.3	Matriz de rasgos del léxico	Dependencia de scikit-learn
SQLite (sqlite3)	3.45	Repositorio histórico: reseñas, semanas, alertas, reportes	Frente a PostgreSQL: no requiere servidor y el archivo queda como evidencia de cada corrida
joblib	1.6	Guardar y cargar el modelo entrenado	Formato estándar de scikit-learn
pandas	2.3	Tablas y gráficos de la interfaz	Integración directa con Streamlit
Streamlit	1.66	Interfaz web y despliegue (opción A)	Frente a Flask: interfaz en un solo archivo y hosting gratuito en Streamlit Community Cloud
Mermaid CLI	11.16	Diagramas de secuencia UML	Frente a draw.io: el diagrama es texto versionado junto al código
Graphviz	2.43	Diagrama de componentes	Distribución automática de nodos
docx (Node.js)	9.7	Generar este documento con los resultados leídos de los logs	Las cifras del documento salen del mismo archivo que las pruebas
Claude (Anthropic)	Opus 5.5	Asistente de programación y redacción durante el desarrollo (ver bitácora de IA)	No interviene en tiempo de ejecución

Qué hace el código propio y qué hace cada herramienta. El código propio hace la percepción, la validación, la planificación, la comparación estadística, la decisión de alertar, la redacción de la alerta, el reporte y la gestión de la memoria. scikit-learn solo entrega la probabilidad de cada polaridad y cada tema para un texto. SQLite solo guarda y devuelve datos. No se usa ningún modelo de lenguaje grande: ninguna decisión del agente depende de un LLM.

9.3 Entradas (inputs)

Ejemplos tomados del log de la corrida E02, semilla 11 (salidas/pruebas/corridas/E02_s11).

Percepción	Tipo de dato	Rango o formato	Origen	Ejemplo real
id	Texto	R00001–R99999	API del conector	R00344
texto	Texto UTF-8	0 a ~1.000 caracteres; puede traer HTML, emojis o venir vacío	API del conector	«Había cucarachas en el cuarto, un asco. Pedimos aseo varias veces y nadie apareció. Qué decepción.»
plataforma	Categórico	{TripAdvisor, Google}	API del conector	TripAdvisor
fecha	Fecha y hora ISO 8601	Dentro de las 14 semanas simuladas; puede ser nula	API del conector	2026-03-10T02:05:37
calificacion	Entero o nulo	1 a 5; nulo en ~15 % de TripAdvisor	API del conector	1
disponibilidad	Booleano por plataforma	true / false	Resultado de la consulta	{TripAdvisor: true, Google: true}
historial	Registros JSON por semana	Hasta 6 semanas previas; conteos por tema y polaridad	Repositorio SQLite	semana 8: limpieza {negativa: 2, neutra: 3, positiva: 6}
configuracion	Números reales y enteros	alfa 0,001–0,2; delta 0–0,5; mínimo de negativas 1–10; confianza 0,3–0,9	config.json o interfaz	alfa 0,08; delta 0,25; mínimo 6; confianza 0,55
reloj	Fecha y hora	Cada 6 h	Bucle de ejecución	2026-03-16T12:00:00

9.4 Salidas y acciones

Acción (método)	Efecto en el entorno	Ejemplo real (E02, semilla 11)
_a_clasificar_lote()	Fila en la tabla resenas con polaridad, tema y confianza	R00344 → negativa, limpieza, confianza 0,998
_a_limpiar_lote()	Fila en la tabla descartadas con el motivo	R00002 → idioma_pt
_a_decidir_alerta()	Mensaje en bandeja_salida.jsonl (o correo SMTP)	«las reseñas negativas sobre limpieza subieron a 70 % (8 de 12), frente a un promedio de 26 % en las últimas 6 semanas…»
_a_generar_reporte()	reportes/semana_NN.md y .json	semana_09.md generado el lunes 16/03 a las 12:00, 12 h después del cierre
_a_registrar_historial()	Fila en la tabla semanas; la semana pasa a ser línea base	semana 9 marcada con alerta (se excluye de la línea base siguiente)
_a_reevaluar_semanas_tardias(	Alerta de tipo «tardía» si las reseñas	Ver escenario E11

Acción (método)	Efecto en el entorno	Ejemplo real (E02, semilla 11)
)	atrasadas cambian la decisión	
esperar (decisión)	Ningún cambio; queda registrado el motivo	«esperar margen de reseñas tardías (6 h de 12 h)»

Cada percepción, decisión y acción queda en log_ejecucion.jsonl con la marca de tiempo real y la simulada. Ejemplo de la decisión que cierra la semana 9:

```
{"ts_simulado": "2026-03-16T12:00:00", "fase": "DECISION",
 "evento": "plan", "semana": 9, "motivo": "plan completo",
 "meta": "mantener_conteos_al_dia+entregar_resumen_semanal",
 "acciones": ["limpiar_lote", "clasificar_lote",
   "actualizar_conteos", "comparar_historial", "decidir_alerta",
   "generar_reporte", "registrar_historial"],
 "hechos": ["lote_nuevo", "puede_cerrar", "semana_cerrada"]}
```

9.5 Diagramas de secuencia UML

La figura 2 muestra el escenario típico: el ciclo de cada seis horas (loop), la clasificación del lote y, al cierre de la semana, la decisión de alertar o no (alt). La figura 3 muestra el escenario de excepción: TripAdvisor no responde al cierre y el agente espera, reintenta y, si se acaba el plazo, entrega el reporte marcado como incompleto. Las fuentes están en docs/secuencia_tipico.mmd y docs/secuencia_excepcion.mmd; los nombres de los mensajes son los de los métodos del código.

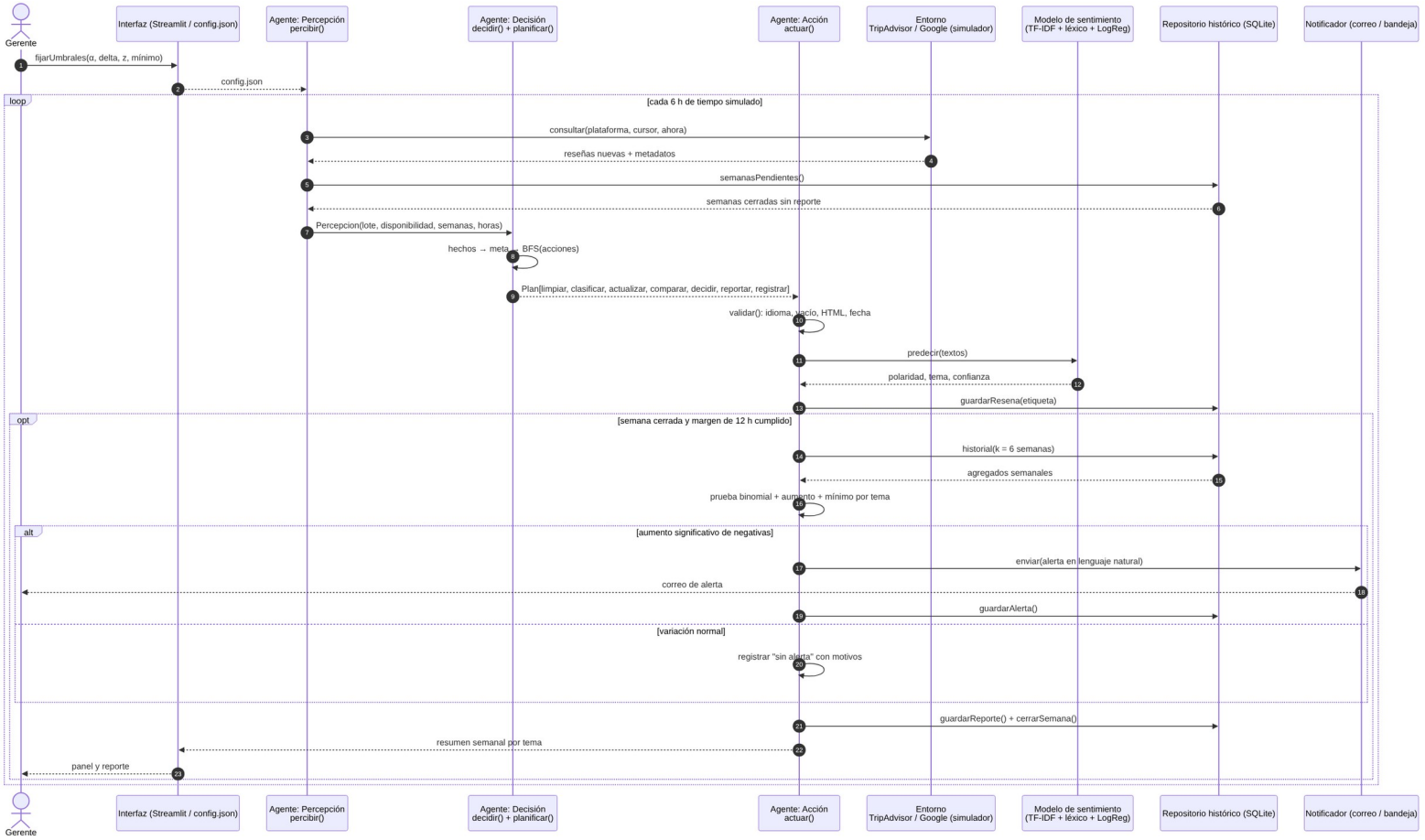


Figura 2. Diagrama de secuencia, escenario típico.

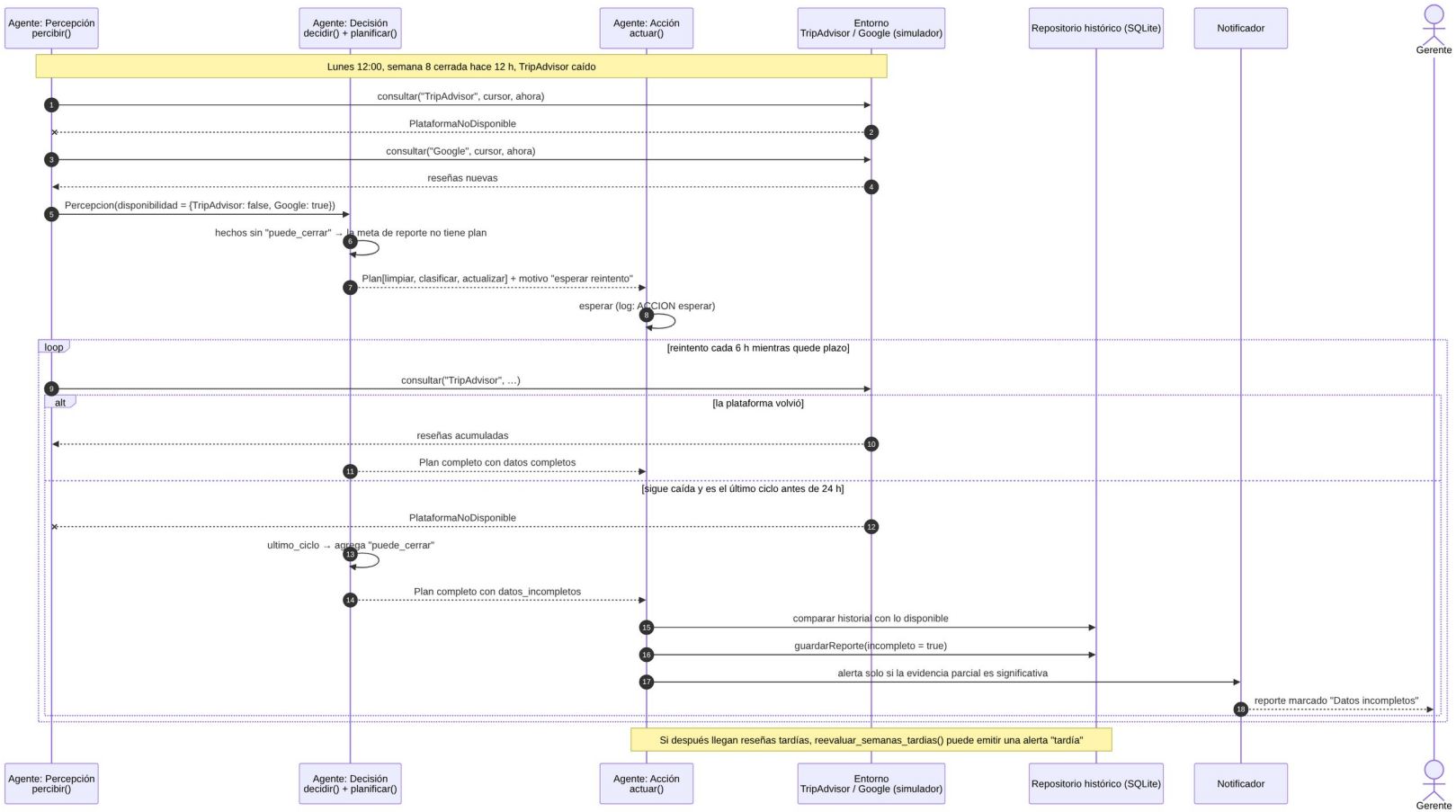


Figura 3. Diagrama de secuencia, escenario de excepción (plataforma caída al cierre).

9.6 Diferencias entre diseño e implementación

Aspecto	Diseño	Implementación	Por qué
Fuente de reseñas	TripAdvisor y Google reales	Simulador con conectores de la misma forma	Sin acceso legal a las API para el curso; el simulador además conoce la verdad, lo que permite medir M2 y M4
Modelo de sentimiento	BERT ajustado para español (IberLEF 2021)	TF-IDF + léxico + regresión logística; clase ClasificadorBERT como punto de reemplazo	El entorno donde se construyó y probó el agente (el entorno de trabajo del asistente de IA) no tenía GPU ni acceso a Hugging Face; se conserva la interfaz para cambiarlo
Datos de entrenamiento	Reseñas reales etiquetadas	Corpus sintético de 4.000 reseñas generado con plantillas y ruido	El corpus de IberLEF se entrega bajo solicitud y no se usó en esta entrega
Regla de alerta	«comparación estadística frente al historial»	Prueba binomial con Bonferroni (α = 0,08 global), aumento ≥ 25 puntos y ≥ 6 negativas; parámetros calibrados con semillas 101–105	Concretar la regla; calibrar sin tocar las semillas de prueba
Reseñas ambiguas	«tratamiento distinto»	Confianza < 0,55: cuentan 0,5 y se marcan para revisión manual	Concretar el tratamiento
Reseñas tardías	No contemplado	Reevaluación de semanas cerradas y alerta tardía	El ambiente es dinámico
Notificación	Correo o mensajería	Bandeja JSONL; SMTP opcional	Evitar enviar correos reales en las pruebas

9.7 Pruebas

Los umbrales se fijaron el 15 de octubre de 2026 en pruebas/umbrales.json antes de la primera ejecución completa. Cada medida se probó con al menos 10 casos, incluyendo casos límite y al menos dos de falla esperada. Como el entorno y el entrenamiento tienen aleatoriedad, cada caso se repitió 5 veces con las semillas 11, 22, 33, 44, 55 y se reportan media y desviación estándar. En M1, la repetición consiste en reentrenar el modelo con otra semilla; en M2–M4, en generar otro flujo de reseñas. Para ser transparentes: una corrida preliminar con la semilla 11 mostró falsos positivos por la negación mal manejada, y eso motivó la corrección del léxico; desde entonces, los parámetros de decisión se calibraron solo con las semillas 101 a 105 (pruebas/calibrar_umbrales.py). El conjunto de control C03 se evaluó una vez durante el desarrollo y no se ajustó nada a partir de él. Los conjuntos que las tablas llaman «manuales» (C02 y C03, 40 reseñas cada uno) se redactaron aparte del corpus de entrenamiento con asistencia de IA; no son reseñas reales. Comando: python pruebas/ejecutar_pruebas.py. El modo --rapido (una sola semilla) escribe en salidas/pruebas_rapido/ y no sustituye estos resultados: con una semilla, la polaridad de C03 puede quedar justo en 85,0 %.

Tabla de resultados

Medida	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia
M1 Exactitud de clasificación	aciertos / reseñas evaluadas, por	≥ 85 % en ambas	11 casos (3 normales, 6 límite, 2	Control C03 (n = 40): polaridad 81,0 % ± 2,9 %;	No	m1_clasificacion.csv; minuto 14:00

Medida	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia
	separado para polaridad y tema		de falla) × 5 reentrenamientos	tema 90,0 % ± 0,0 %		
M2 Detección de cambios en 24 h	incidentes relevantes alertados ≤ 24 h tras el cierre / incidentes relevantes	≥ 90 %	12 escenarios × 5 semillas = 60 corridas; 50 incidentes	52,0 % (26/50); por corrida ± 48,9 %	No	escenarios.csv; corridas/*/log_ejecucion.jsonl; minuto 16:30
M3 Reporte semanal a tiempo	reportes ≤ 24 h tras el cierre / semanas cerradas	100 %	60 corridas; 840 semanas	100,0 % (840/840); latencia máxima 24 h	Sí	corridas/*/reportes/; minuto 10:30
M4 Falsos positivos de alerta	alertas sin incidente relevante / alertas enviadas	< 10 %	60 corridas; 32 alertas	3,1 % (1/32); por corrida ± 18,3 %	Sí	corridas/*/bandeja_salida.jsonl; minuto 16:30

Detalle de M1 por caso

Caso	Descripción	Tipo	n	Polaridad	Tema	Rechazo	¿Cumple?
C01	Sintético con frases no vistas	normal	500	88,7 % ± 1,5 %	89,2 % ± 1,0 %	—	Sí
C02	Manual de desarrollo (40)	normal	40	87,5 % ± 1,8 %	97,0 % ± 2,7 %	—	Sí
C03	Manual de control, nunca usado al ajustar (40)	normal	40	81,0 % ± 2,9 %	90,0 % ± 0,0 %	—	No
C04	Control en minúsculas y sin tildes	limite	40	81,0 % ± 2,9 %	90,0 % ± 0,0 %	—	No
C05	Control con errores de tipeo	limite	40	70,0 % ± 5,0 %	84,0 % ± 2,9 %	—	No
C06	Solo ironía y sarcasmo (12)	limite	12	100,0 % ± 0,0 %	100,0 % ± 0,0 %	—	Sí
C07	Reseñas largas (>60 palabras)	limite	300	81,5 % ± 0,9 %	100,0 % ± 0,0 %	—	No
C08	Reseñas de una sola frase	limite	300	59,7 % ± 3,3 %	89,3 % ± 1,5 %	—	No
C09	Sintético con 30 % de ironía	limite	500	88,1 % ± 1,4 %	90,2 % ± 1,1 %	—	Sí
C10	Otro idioma: debe descartarse	falla	7	—	—	100,0 % ± 0,0 %	Sí
C11	Entradas inválidas: vacías, emojis, HTML, sin fecha	falla	8	—	—	100,0 % ± 0,0 %	Sí

Detalle de M2, M3 y M4 por escenario

Escenario	Tipo	Detección ≤ 24 h	Alertas tardías	Reportes a tiempo	Alertas	FP (media ± d. e. por corrida)
E01 Operación estable	normal	sin incidente	0	100,0 % ± 0,0 %	0	0 (0,0 ± 0,0)
E02 Incidente de limpieza	normal	60,0 % ± 54,8 %	0	100,0 % ± 0,0 %	3	0 (0,0 ± 0,0)
E03 Dos incidentes	normal	70,0 % ± 27,4 %	0	100,0 % ± 0,0 %	7	0 (0,0 ± 0,0)
E04 Temporada alta	limite	sin incidente	0	100,0 % ± 0,0 %	1	1 (0,2 ± 0,4)
E05 Incidente moderado	limite	40,0 % ± 54,8 %	0	100,0 % ± 0,0 %	2	0 (0,0 ± 0,0)
E06 Variación leve	limite	sin incidente	0	100,0 % ± 0,0 %	0	0 (0,0 ± 0,0)

Escenario	Tipo	Detección ≤ 24 h	Alertas tardías	Reportes a tiempo	Alertas	FP (media ± d. e. por corrida)
E07 Volumen bajo	limite	0,0 % ± 0,0 %	0	100,0 % ± 0,0 %	0	0 (0,0 ± 0,0)
E08 Ironía alta	limite	100,0 % ± 0,0 %	0	100,0 % ± 0,0 %	5	0 (0,0 ± 0,0)
E09 Entradas inválidas	falla	80,0 % ± 44,7 %	0	100,0 % ± 0,0 %	4	0 (0,0 ± 0,0)
E10 TripAdvisor caído	falla	100,0 % ± 0,0 %	0	100,0 % ± 0,0 %	5	0 (0,0 ± 0,0)
E11 Todas las plataformas caídas	falla	0,0 % ± 0,0 %	5	100,0 % ± 0,0 %	5	0 (0,0 ± 0,0)
E12 Arranque en frío	falla	0,0 % ± 0,0 %	0	100,0 % ± 0,0 %	0	0 (0,0 ± 0,0)

Sin los cuatro escenarios de falla esperada, la detección es 56,7 % (17/30) y los falsos positivos 5,6 % (1/18).

Análisis de resultados

Se cumplen M3 y M4. El agente entregó 840 de 840 reportes dentro de 24 horas, incluidas las semanas con plataformas caídas, porque la planificación por metas cambia a «cerrar con datos incompletos» en el último ciclo antes del plazo. Solo 1 de 32 alertas fue un falso positivo (3,1 %), por debajo del 10 %: el gerente puede confiar en lo que recibe. El único falso positivo ocurrió en temporada alta (E04).

No se cumple M1. En el conjunto de control, que no se usó para ajustar nada, la polaridad llega a 81,0 % ± 2,9 % y el tema a 90,0 %; el tema cumple, la polaridad no. En frases sintéticas no vistas (C01) el modelo sí pasa el umbral (88,7 %), lo que muestra la causa: se entrenó con un corpus sintético de vocabulario limitado y generaliza mal a expresiones reales como «tardaron muchísimo» o «cobran todo aparte». Las reseñas de una sola frase (C08, 59,7 %) y con errores de tipeo (C05, 70,0 %) son los casos más débiles, y de los ocho errores del modelo base en ese conjunto, seis involucran la clase neutra y cinco son reseñas negativas que el modelo no reconoce como tales, lo que también le resta detección a M2. El 100 % de C06 (ironía) no es informativo: esas frases están en el corpus de entrenamiento. C04 tampoco es una prueba independiente: el preprocesamiento (normalizar) ya pasa todo a minúsculas y quita las tildes antes del modelo, por eso da exactamente lo mismo que C03. Además, con 40 reseñas cada una pesa 2,5 puntos y el margen de error de un 81 % es de unos ±12 puntos, así que este conjunto es demasiado pequeño para decidir con seguridad frente a un umbral de 85 %; el veredicto «No» se apoya en el promedio de cinco reentrenamientos y en que C05, C07 y C08 también quedan por debajo. **Corrección:** reemplazar el modelo por uno preentrenado para español (la clase ClasificadorBERT ya deja el punto de cambio) y entrenarlo con reseñas reales etiquetadas, como el corpus de TripAdvisor de IberLEF 2021.

No se cumple M2: 52,0 % (26/50) frente a 90 %. Diez de los 24 incidentes perdidos son fallas esperadas por diseño: en E11 no hay datos al cierre (las 5 alertas llegan tarde, después de 24 h) y en E12 no hay historial con qué comparar. Sin los escenarios de falla, la detección es 56,7 %. En E07 (12 reseñas por semana) no detecta nada porque el mínimo de 6 negativas es inalcanzable con unas 5 reseñas por tema; en E02, E03 y E05 falla cuando el incidente real produce pocas reseñas negativas en la semana (en una semilla de E02, solo 11 de las 23 reseñas de limpieza de esa semana se clasificaron como negativas).

Para separar causas se aplicó la misma regla a las etiquetas reales del simulador (pruebas/techo_deteccion.py). Con un clasificador perfecto, la detección en los escenarios normales y límite sería 66,7 %, no 90 %; con el doble de volumen semanal subiría a 86,7 %. Es decir, el clasificador explica unos 10 puntos de la diferencia y el resto es potencia estadística: con unas 9 reseñas por tema a la semana, una ventana semanal no alcanza para distinguir un incidente de la variación normal sin disparar falsos positivos. **Corrección:** (1) usar una prueba secuencial acumulada (por ejemplo, CUSUM diario) en lugar de cortes semanales, para acumular evidencia en varios días; (2) fusionar la calificación numérica con el texto, que reduce el ruido del clasificador; (3) escalar el mínimo de negativas con el volumen del tema; y (4) reformular M2 para hoteles de bajo volumen como detección dentro de 48 h o con ventanas de 14 días.

9.8 Limitaciones y trabajo futuro

- Los conjuntos de evaluación manuales (C02 y C03) y el corpus de entrenamiento se redactaron con asistencia de IA; ninguno contiene reseñas reales. La exactitud medida es una estimación optimista de lo que pasaría con lenguaje real, y el siguiente paso es evaluar con reseñas reales etiquetadas por personas.
- El entorno es simulado. El agente no se ha probado con reseñas reales de TripAdvisor o Google; el simulador reproduce las propiedades del ambiente, pero su lenguaje es más limitado que el real.
- El modelo de sentimiento se entrenó con un corpus sintético. Su exactitud en lenguaje natural (conjunto de control) queda por debajo de la meta; es la primera mejora: un modelo preentrenado para español y datos reales etiquetados.
- La detección semanal pierde incidentes en hoteles de bajo volumen. Trabajo futuro: prueba secuencial diaria y umbrales que se ajusten al volumen.
- La calificación numérica se recoge, pero no se usa para decidir. Trabajo futuro: combinarla con la probabilidad del texto.
- No hay aprendizaje en línea. Trabajo futuro: usar las reseñas que el gerente corrija en la revisión manual para reentrenar el modelo cada mes, con validación antes de reemplazarlo.
- Las alertas se escriben en una bandeja de salida; el envío por correo SMTP está implementado pero no se probó con un servidor real.
- El agente atiende un solo hotel. Para varios hoteles habría que separar el historial por hotel y revisar la corrección por comparaciones múltiples.

9.9 Enlaces

- Repositorio: [pegar enlace de GitHub]
- Plataforma de despliegue (Streamlit Community Cloud): [pegar enlace de Streamlit Community Cloud]
- Video (≤ 20 min): [pegar enlace de YouTube no listado]
- Logs de las pruebas: salidas/pruebas/ en el repositorio (m1_clasificacion.csv, escenarios.csv, resumen.json, corridas/*/log_ejecucion.jsonl)
- Bitácora de uso de IA: docs/bitacora_uso_ia.docx

10. Conclusiones

ReviewPulse-Agent se construyó como un agente basado en modelos y orientado a metas, coherente con el diseño: mantiene un estado interno en dos plazos, planifica sus acciones con búsqueda en anchura sobre hechos y metas, y su ciclo percepción → decisión → acción se puede seguir en el código y en el log de cada ejecución. El agente funciona de punta a punta en los doce escenarios, incluidos los de falla, sin detenerse por entradas inválidas o plataformas caídas.

De las cuatro medidas de desempeño, se cumplen dos: el reporte semanal llega a tiempo en el 100 % de las semanas y los falsos positivos quedan en 3,1 %. No se cumplen la exactitud de polaridad en lenguaje real (81,0 % frente a 85 %) ni la detección de incidentes en 24 horas (52,0 % frente a 90 %). El análisis muestra que la detección está limitada sobre todo por el volumen de reseñas por semana: ni siquiera un clasificador perfecto llegaría al 90 % con la ventana semanal. Ese es el hallazgo más útil de las pruebas, porque indica que la siguiente versión debe cambiar la forma de acumular evidencia, no solo el modelo.

En lo ético se mantiene lo planteado en el diseño: las reseñas son públicas, pero las escribieron personas reales, y el modelo puede tener sesgos hacia ciertos estilos de escritura. Por eso el agente marca para revisión humana las clasificaciones de baja confianza, explica cada alerta con cifras y ejemplos, y deja la decisión final en manos del gerente.

Referencias

IBM. (s. f.). Components of AI agents. IBM Think. https://www.ibm.com/mx-es/think/topics/components-of-ai-agents

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4.ª ed., Global Edition). Pearson.

Streamlit Inc. (2026). Streamlit documentation. https://docs.streamlit.io

Vásquez, J., Gómez-Adorno, H., & Bel-Enguix, G. (2021). BERT-based approach for sentiment analysis of Spanish reviews from TripAdvisor. En Proceedings of the Iberian Languages Evaluation Forum (IberLEF 2021), CEUR Workshop Proceedings, 2943, 165–170. https://ceur-ws.org/Vol-2943/

Wasserman, L. (2004). All of Statistics: A Concise Course in Statistical Inference. Springer. (Pruebas de hipótesis y corrección de Bonferroni, cap. 10).